

DBI Dokumentation Bet Fanaticos

Projektbeschreibung

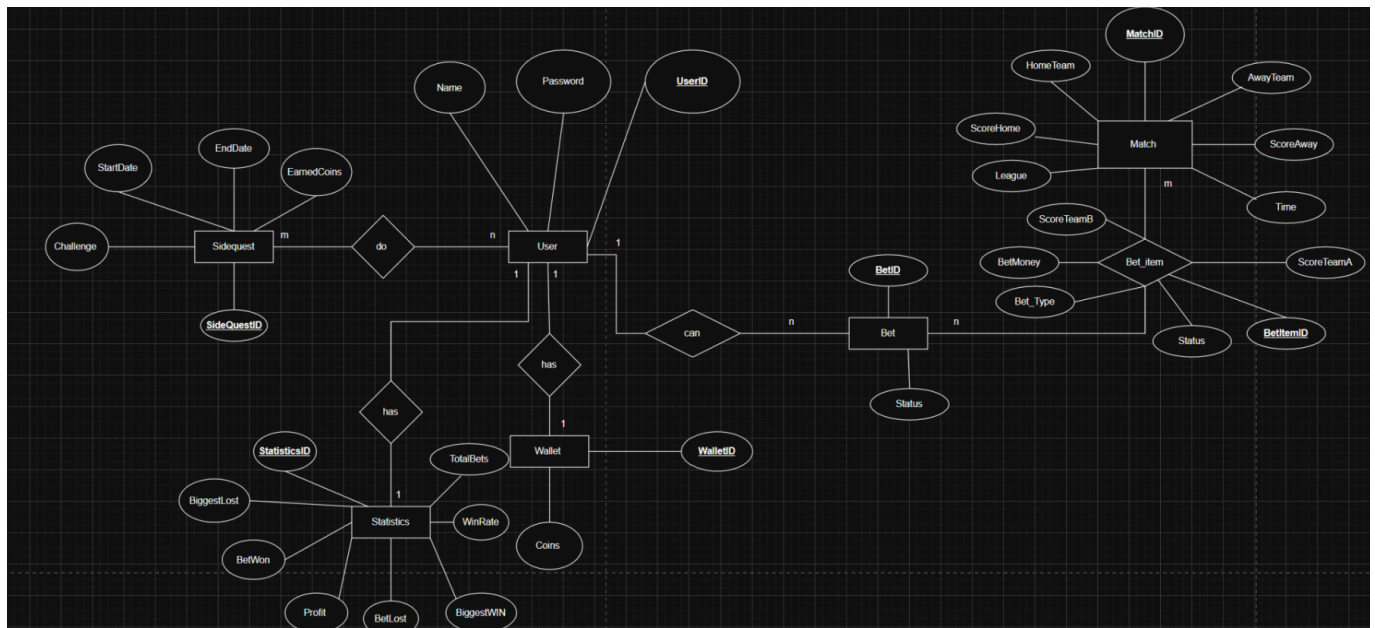
Die WPF-Anwendung ist eine Wettsimulation und dient ausschließlich zu Bildungszwecken. Sie soll verdeutlichen, welche Auswirkungen Sportwetten haben und wie sich Wettverhalten auf Spieler auswirken kann.

Im Spiel erhalten die Nutzer eine bestimmte Anzahl an Wett-Coins, mit denen sie virtuelle Wetten platzieren können. Täglich werden rund 100 Spiele aus verschiedenen Sportarten wie Fußball, Basketball und Volleyball angezeigt, auf die gewettet werden kann.

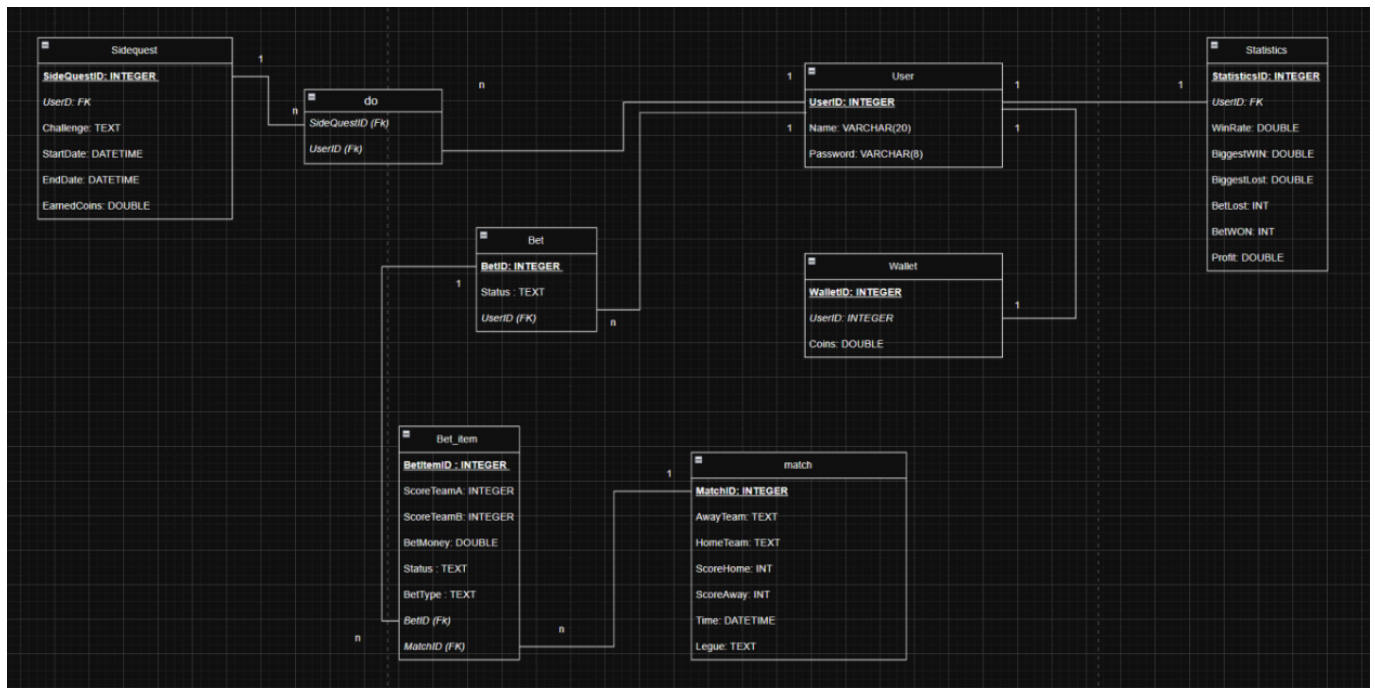
Zusätzlich gibt es tägliche Challenges, die abgeschlossen werden können, um weitere Wett-Coins zu verdienen. Diese Herausforderungen sorgen für zusätzliche Motivation und Abwechslung innerhalb der Simulation.

Ziel der Anwendung ist es, ein realistisches Wettgefühl zu vermitteln, ohne dass dabei echtes Geld eingesetzt wird.

ERM



RM



Normalisierungsnachweis

1 NF

- Wird erfüllt, weil Attribute Atomar vorliegen
- Alle Tabellen haben Key-Pairs
- Datentypen sind vordefiniert

2 NF

- 1NF wird erfüllt
- Jedes Nicht-Schlüssel-Attribut ist von jedem Teil des Primärschlüssels voll funktional abhängig

3 NF

- 2NF wird erfüllt
- Ist in der 3. Jetzt, weil die attributen für statistiken aggregiert werden und somit die abhängigkeiten wegfallen

Projekttagbuch

Kenan

Datum	Beschreibung
02.06.2026	Zusätzliche Datenbanktabellen und API-Routen für Wetten, Wettpositionen, Wallets und Sidequests wurden implementiert.
08.06.2026	Der Abruf der Sportdaten wurde vom C#-Client in das FastAPI-Backend verlagert. Zusätzlich wurden Fehler in den Modellen und in der Datei <code>main.py</code> behoben.
10.06.2026	Zusätzlich Basketball API implementiert

Datum	Beschreibung
17.06.2026	Das Wallet-System wurde erweitert, sodass die Coins der Benutzer in der Datenbank gespeichert und wieder geladen werden können.
17.06.2026	Die Teamstärke wurde in das System integriert, um später Wettquoten berechnen zu können.
18.06.2026	Die Wettlogik wurde erweitert. Jede Wette erhält eine eindeutige ID und kann dadurch eindeutig gespeichert und verarbeitet werden.
21.06.2026	Endpunkte in Pycharm erklärt durch Kommentare

Emir

Datum	Beschreibung
20.05.2026	Die grundlegende Projektstruktur wurde eingerichtet.
28.05.2026	Die Ordnerstruktur wurde erweitert. Zusätzlich wurden Router-Ordner, Requirements sowie erste Datenbankbestandteile wie Datenmodell, Engine und Session angelegt.
01.06.2026	Erste Datenbanktabellen wurden erstellt.
10.06.2026	API-Pfade wurden aktualisiert bzw. angepasst.
14.06.2026	Die Authentifizierung wurde fertiggestellt und Benutzerrollen wie admin und user wurden hinzugefügt.
16.06.2026	Die Authentifizierung wurde weiter stabilisiert und in den Hauptstand integriert. Zusätzlich wurde ein Sicherheitscommit erstellt.
17.06.2026	Der Datenbankpfad wurde dynamisch gesetzt, statt einen absoluten Pfad zu verwenden.
21.06.2026	Logging wurde umgesetzt und mit dem Chat bzw. der Aggregation verbunden.

Bedienungsanleitung

Starten der API

Zum Starten der API muss lediglich die Datei **main.py** ausgeführt werden. Anschließend startet der FastAPI-Server und die Endpunkte sind verfügbar.

Die API ist danach unter folgender Adresse erreichbar:

```
http://127.0.0.1:8000
```

Die automatische Dokumentation kann über folgende Adresse aufgerufen werden:

```
http://127.0.0.1:8000/docs
```

Beispiel-Requests

Alle Matches abrufen

```
GET /match
```

Alle Sidequests abrufen

```
GET /sidequest
```

Admin-Rechte

Mit dem Make-Admin-Router kann ein normaler User zu einem Admin gemacht werden. Dieser Router ist jedoch nur für das initiale Setup gedacht. Dafür kann die Dependency Injection, die Admin-Rechte voraussetzt, kurzzeitig entfernt und danach wieder eingesetzt werden.

In einer produktiven Umgebung wäre dieser Router nicht öffentlich erreichbar. Er wird nur am Anfang verwendet, um den ersten Admin zu erstellen. Danach wird er deaktiviert oder wieder geschützt. Theoretisch darf nur der Publisher beziehungsweise Entwickler diesen Schritt durchführen.

Statistiken

Die Statistik ist bereits in Swagger UI sichtbar. Hier ist trotzdem eine kurze Erklärung des Ablaufs:

Zuerst muss ein Dummy-Match erstellt werden. Danach kann ein User mit dem Bet-Router auf dieses Match wetten. Dabei wird automatisch ein Eintrag in bets und ein Eintrag in betitem erstellt.

bets speichert, welcher User gewettet hat. betitem speichert die Details der Wette, zum Beispiel match_id, Einsatz, Quote, Prediction und Status. Der separate Betitem-Router wird nur für Debug-Zwecke verwendet und ist nicht für den normalen Wettablauf gedacht.

Im Statistics-Router kann anschließend ein Ergebnis für das Match eingetragen werden, zum Beispiel score_home und score_away. Dadurch wird geprüft, ob die Prediction des Users richtig oder falsch war. Danach kann über den Recalculate-Endpoint die Statistik neu berechnet und in der statistics-Tabelle gespeichert werden.

Challenges

Die Challenges wurden hauptsächlich für den POS-Teil verwendet. Dafür wurden Dummy-Challenges gespeichert und den Usern zugeteilt.

Jeder User hat dabei seinen eigenen Fortschritt.

Dadurch kann jeder User unabhängig von anderen Usern eine Challenge starten, fortsetzen und abschließen.

Änderungen während Projekt

Web-API

Die ursprüngliche Idee war, die Statistik im C#-Client zu berechnen. Das wäre jedoch ein unnötiger Umweg, weil dafür Daten zuerst aus der Datenbank geladen, im Client aggregiert und anschließend wieder zurückgeschickt werden müssten. Deshalb wird die Statistik direkt im Backend berechnet und gespeichert.

RM nach Umsetzung

